Модуль 7

PostgreSQL и Prisma ORM

PostgreSQL • Prisma • Схема • Миграции • CRUD • Связи • Фильтрация • Транзакции

7.1 SQL vs NoSQL — когда что использовать

В предыдущем модуле мы изучили MongoDB — NoSQL базу данных. Теперь рассмотрим реляционную базу PostgreSQL. Выбор между ними зависит от задачи:

PostgreSQL (SQL) подходит, когда:

- Данные имеют чёткую, стабильную структуру (финансы, e-commerce, CRM)
- Нужны сложные запросы с JOIN, агрегацией, подзапросами
- Важна целостность данных (транзакции ACID, внешние ключи)
- Нужна строгая типизация и валидация на уровне БД

MongoDB (NoSQL) подходит, когда:

- Структура данных часто меняется (стартапы, прототипы)
- Данные естественно вложенные (посты с комментариями, каталоги)
- Нужно горизонтальное масштабирование
- Работа с большими объёмами неструктурированных данных

На практике многие проекты используют несколько баз данных: PostgreSQL для основных данных, Redis для кеша, MongoDB для логов.

SQL и NoSQL: два подхода к хранению данных

В модуле 6 мы работали с MongoDB, документной NoSQL-базой. Теперь познакомимся с реляционными (SQL) базами данных на примере PostgreSQL.

Реляционные базы данных (SQL)

Данные хранятся в таблицах со строго определённой структурой (столбцы и строки). Между таблицами устанавливаются связи через внешние ключи (foreign keys). Примеры: PostgreSQL, MySQL, SQLite.

Ключевые особенности SQL:

- Строгая схема: каждая таблица имеет фиксированный набор столбцов с типами данных.
- ACID-транзакции: гарантия целостности данных (атомарность, согласованность, изоляция, долговечность).
- SQL-запросы: мощный язык для выборки, фильтрации, агрегации и объединения данных (JOIN).
- Нормализация: данные не дублируются, хранятся в связанных таблицах.

Документные базы данных (NoSQL)

Данные хранятся в виде документов (JSON-объектов) без строгой схемы. Примеры: MongoDB, CouchDB.

Ключевые особенности NoSQL:

- Гибкая схема: каждый документ может иметь разный набор полей.

- Горизонтальное масштабирование: легко распределять данные между серверами.

- Нет JOIN: связи реализуются через вложенные документы или ссылки (populate).

Сравнительная таблица

SQL (PostgreSQL):

- Структура: таблицы, строки, столбцы.
- Схема: строгая, задаётся заранее.
- Связи: JOIN, внешние ключи.
- Транзакции: полноценные ACID.
- Язык запросов: SQL.
- Подходит: финансы, e-commerce, CRM, любые данные со сложными связями.

NoSQL (MongoDB):

- Структура: коллекции, документы (JSON).
- Схема: гибкая, может меняться.
- Связи: вложенные документы, populate.
- Транзакции: поддерживаются, но ограниченно.
- Язык запросов: методы Mongoose / MongoDB Query.
- Подходит: контент, логи, IoT, прототипы, часто меняющаяся структура.

Когда выбрать PostgreSQL?

- Данные имеют чёткую структуру и сложные связи (пользователи, заказы, товары, категории).

- Нужны надёжные транзакции (финансовые операции, бронирование).

- Требуется мощная фильтрация, агрегация, полнотекстовый поиск.

- Важна целостность данных (нельзя удалить автора, пока есть его книги).

Итак, что такое PostgreSQL?

PostgreSQL (или просто Postgres) это мощная реляционная база данных с открытым исходным кодом. Она поддерживает JSON-поля, полнотекстовый поиск, массивы, enum-типы и многое другое. PostgreSQL является одной из самых популярных баз данных для серверных приложений.

7.2 Установка PostgreSQL

PostgreSQL так же как и другие БД, может быть установлена несколькими способами.

Вариант 1. Установка в Docker (рекомендуется для разработки)

Самый простой способ запустить PostgreSQL в изолированном контейнере без установки на основную систему. Это удобно: не засоряется ОС, легко удалить и пересоздать базу.

Убедитесь, что Docker установлен:

```
docker --version
```

Если Docker не установлен, скачайте его с https://www.docker.com/products/docker-desktop/

Скачайте официальный образ PostgreSQL:

```
docker pull postgres
```

Запустите контейнер PostgreSQL с пользователем и паролем.

POSTGRES_USER, POSTGRES_PASSWORD и POSTGRES_DB – это переменные окружения, которые передаются в контейнер при его запуске.

```
docker run -d \
  --name postgres_db \
  -p 5432:5432 \
  -e POSTGRES_USER=pguser \
  -e POSTGRES_PASSWORD=pgpass \
  -e POSTGRES_DB=myapp \
  -v pgdata:/var/lib/postgresql \
  postgres
```

Разбор параметров:

-d: запуск в фоновом режиме.

--name postgres_db: имя контейнера.

-p 5432:5432: проброс порта PostgreSQL.

-e POSTGRES_USER=pguser: имя пользователя.

-e POSTGRES_PASSWORD=pgpass: пароль.

-e POSTGRES_DB=myapp: имя базы данных (создаётся автоматически).

- -v pgdata:/var/lib/postgresql/data: том для хранения данных.

Проверьте, что контейнер запущен:

```
docker ps
```

Вы должны увидеть контейнер с именем postgres_db и статусом Up.

Строка подключения для Docker-контейнера с авторизацией:

```
postgresql://pguser:pgpass@localhost:5432/myapp
```

Полезные команды Docker:

```
docker stop postgres_db  # остановить контейнер
docker start postgres_db # запустить снова
docker logs postgres_db  # посмотреть логи
docker exec -it postgres_db psql -U pguser -d myapp # войти в командную
строку контейнера
docker rm -f postgres_db # удалить контейнер (данные сохранятся в томе
pgdata)
docker volume rm pgdata  # удалить том с данными (необратимо!)
```

Вариант 2. Локальная установка

Установка локально выполняется разными способами в зависимости от операционной системы.

Установка на macOS

Установите Homebrew (если ещё не установлен): https://brew.sh

Выполните команды в консоли

```
brew install postgresql@16
brew services start postgresql@16
```

Создание пользователя и базы данных:

```
sudo -u postgres psql
CREATE USER pguser WITH PASSWORD 'pgpass';
CREATE DATABASE myapp OWNER pguser;
\q
```

Проверка подключения:

```
psql -U pguser -d myapp -h localhost
```

Если видите приглашение myapp=> PostgreSQL работает.

Установка на Windows

Скачайте установщик с https://www.postgresql.org/download/windows/

В процессе установки задайте пароль для пользователя postgres. После установки PostgreSQL запустится как служба Windows.

Linux (Ubuntu / Debian)

Выполните команды:

```
sudo apt update
sudo apt install -y postgresql postgresql-contrib
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

Создание пользователя и базы данных:

```
sudo -u postgres psql
CREATE USER pguser WITH PASSWORD 'pgpass';
CREATE DATABASE myapp OWNER pguser;
\q
```

Проверка подключения:

```
psql -U pguser -d myapp -h localhost
```

Если видите приглашение myapp=> PostgreSQL работает.

7.3 PrismaORM — ORM для SQL баз данных

ORM (Object-Relational Mapping) — это инструмент, который позволяет работать с таблицами базы данных как с объектами в коде. Вместо написания SQL-запросов вручную вы вызываете методы: create(), findMany(), update(), delete().

В модуле 6 мы использовали Mongoose как ODM для MongoDB. Для PostgreSQL мы будем использовать Prisma ORM. Это современный ORM для Node.js с типобезопасным API, автогенерацией клиента и удобной системой миграций.

Prisma — современный ORM (Object-Relational Mapper) для Node.js и TypeScript. Он отличается от традиционных ORM (Sequelize, TypeORM) декларативной схемой, автоматическими миграциями и полной типобезопасностью.

Prisma состоит из трёх компонентов:

- Prisma Schema — файл, описывающий модели данных и связи
- Prisma Migrate — система миграций (создание и изменение таблиц)
- Prisma Client — автоматически сгенерированный клиент для запросов к БД

Установка Prisma ORM:

Убедитесь, что у вас есть проект с package.json и включён ESM ("type": "module").

Установите пакеты prisma и prisma/client. Prisma устанавливается как devDependency (нужен только для разработки: миграции, генерация клиента). @prisma/client - это runtime-зависимость, которая используется в коде приложения.

```
npm install prisma --save-dev # CLI для миграций и генерации
npm install @prisma/client    # Клиент для запросов
npx prisma init               # Инициализация (создаёт schema.prisma и
.env)
```

При инициализации Prisma создаст файлы prisma/schema.prisma и .env с переменной DATABASE_URL:

Обновите файл .env таким образом

```
DATABASE_URL="postgresql://pguser:pgpass@localhost:5432/myapp"
```

В файл package.json можно добавить скрипты:

```
{
    "scripts": {
        "start": "node --env-file=.env server.js",
        "dev": "node --env-file=.env --watch server.js"
    }
}
```

Скрипт "dev" использует встроенный --watch (Node.js 18+) для автоматической перезагрузки при изменении файлов.

После добавления этих скриптов в package.json вы можете запускать сервер с помощью команды npm run dev

Далее, откройте prisma/schema.prisma и убедитесь, что provider указан правильно:

```
generator client {
    provider = "prisma-client-js"
}


datasource db {
    provider = "postgresql"
    url      = env("DATABASE_URL")
}
```

Блок generator указывает, что Prisma должна сгенерировать JavaScript-клиент. Блок datasource указывает тип базы данных и строку подключения (берётся из переменной окружения).

7.4 Схема данных в Prisma

Схема Prisma описывает модели (таблицы), поля (колонки), типы данных, связи и ограничения. Пример синтаксиса схемы:

```
generator client {
    provider = "prisma-client-js"
}


datasource db {
    provider = "postgresql"   // или "sqlite", "mysql"
    url      = env("DATABASE_URL")
}
# Схема User
model User {
    id        Int      @id @default(autoincrement()) // Первичный ключ
    email     String   @unique                       // Уникальное поле
    name      String
    role      Role     @default(USER)           // Enum с дефолтом
    posts     Post[]                           // Связь один-ко-многим
    profile   Profile?                         // Связь один-к-одному
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt               // Автообновление
}
```

```
# Схема Post
model Post {
    id        Int       @id @default(autoincrement())
    title     String
    content   String?
    published Boolean   @default(false)
    author    User      @relation(fields: [authorId], references: [id])
    authorId  Int
    tags      Tag[]                          // Связь многие-ко-многим
    createdAt DateTime  @default(now())
    updatedAt DateTime  @updatedAt
}
# Схема Tag
model Tag {
    id    Int    @id @default(autoincrement())
    name  String @unique
    posts Post[]                             // Обратная сторона связи
}
# Тип Role
enum Role {
    USER
    ADMIN
    MODERATOR
}
```

Ключевые атрибуты любой схемы:

- @id — первичный ключ
- @unique — уникальное ограничение
- @default(value) — значение по умолчанию (autoincrement(), now(), uuid())
- @updatedAt — автоматически обновляется при изменении записи
- @relation — определяет связь между моделями
- ? после типа — поле необязательное (nullable)
- [] — массив (связь один-ко-многим или многие-ко-многим)

Описание схемы данных нашего приложения

Мы создадим приложение для управления библиотекой книг. Основные сущности: Author (автор) и Book (книга). Автор может написать много книг, а каждая книга принадлежит одному автору (связь один-ко-многим).

Откройте файл prisma/schema.prisma и замените содержимое:

```
generator client {
    provider = "prisma-client-js"
}

datasource db {
    provider = "postgresql"
    url      = env("DATABASE_URL")
}

model Author {
    id        Int      @id @default(autoincrement())
    name      String
    email     String   @unique
    bio       String?
    books     Book[]
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt
}

model Book {
    id        Int      @id @default(autoincrement())
    title     String
    isbn      String   @unique
    pages     Int?
    published Boolean  @default(true)
    author    Author   @relation(fields: [authorId], references: [id])
    authorId  Int
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt
}
```

Разбор модели Author

- id Int @id @default(autoincrement()): первичный ключ, целое число, автоинкремент. В PostgreSQL это столбец типа SERIAL.
- name String: обязательное текстовое поле (в SQL это VARCHAR или TEXT).
- email String @unique: уникальное поле. Prisma создаст уникальный индекс в базе данных.
- bio String?: знак ? означает, что поле необязательное (nullable). В таблице столбец будет допускать NULL.

- books Book[]: связь один-ко-многим. У автора может быть много книг. Это виртуальное поле, оно не создаёт столбец в таблице, а используется Prisma для include/select связанных записей.
- createdAt DateTime @default(now()): дата создания, заполняется автоматически.
- updatedAt DateTime @updatedAt: дата обновления, автоматически обновляется при любом изменении записи.

Разбор модели Book

- title String: название книги (обязательное).
- isbn String @unique: уникальный ISBN.
- pages Int?: количество страниц (необязательное).
- published Boolean @default(false): статус публикации, по умолчанию false.
- author Author @relation(fields: [authorId], references: [id]): связь с моделью Author. Поле authorId хранит ID автора.
- authorId Int: внешний ключ (foreign key). В PostgreSQL Prisma создаст FOREIGN KEY REFERENCES author(id).

Как работает @relation?

В SQL связи между таблицами реализуются через внешние ключи. В схеме Prisma это описывается атрибутом @relation:

```
author   Author @relation(fields: [authorId], references: [id])
authorId Int
```

- fields: [authorId] указывает, какое поле в текущей модели хранит внешний ключ.
- references: [id] указывает, на какое поле в связанной модели ссылается внешний ключ.
- На стороне Author поле books Book[] является обратной стороной связи.

Сравнение с MongoDB/Mongoose

В Mongoose (MongoDB) мы использовали ObjectId и ref для связей:

```
// Mongoose
author: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
```

В Prisma связь описывается явно через @relation с указанием полей:

```
// Prisma
author   Author @relation(fields: [authorId], references: [id])
authorId Int
```

Главное отличие: в SQL внешний ключ это реальное ограничение в базе данных. PostgreSQL не позволит создать книгу с несуществующим authorId, база сама гарантирует целостность данных.

Типы данных в Prisma

Основные типы:

- String: текст (VARCHAR/TEXT в PostgreSQL).
- Int: целое число (INTEGER).
- Float: число с плавающей точкой (DOUBLE PRECISION).
- Boolean: true/false (BOOLEAN).
- DateTime: дата и время (TIMESTAMP).
- Json: JSON-данные (JSONB в PostgreSQL).
- BigInt: большие целые числа (BIGINT).

И еще раз про ключевые атрибуты схемы:

- @id: первичный ключ.
- @unique: уникальное ограничение.
- @default(value): значение по умолчанию (autoincrement(), now(), uuid(), cuid()).
- @updatedAt: автообновление при изменении записи.
- @relation: определяет связь между моделями.
- ? после типа: поле необязательное (nullable).
- [] после модели: массив (связь один-ко-многим или многие-ко-многим).

7.5 Миграции

Миграции — нет, это не переселение животных из одного региона в другой. В программировании — это версионированные изменения структуры базы данных. Каждый раз, когда вы меняете schema.prisma (добавляете модель, поле, связь), нужно создать миграцию. Миграция генерирует SQL-файл, который Prisma применяет к базе данных.

Это похоже на GIT для базы данных. В GIT вы выполняете git add и git commit каждый раз при изменении файлов. А в реляционных базах данных, вы выполняете миграцию, каждый раз при изменении структуры базы во время разработки приложения.

Обратите внимание: в NoSQL миграций нет и не бывает совсем.

Каждая миграция в реляционных базах фиксирует изменение структуры, и вы можете откатиться или посмотреть историю изменений. Prisma позволяет с легкостью создавать миграции.

Создание первой миграции

После того как вы описали модели Author и Book в schema.prisma, выполните:

```
npx prisma migrate dev --name init
```

Эта команда делает три вещи:

1. Создаёт SQL-файл миграции в папке prisma/migrations/<timestamp>_init/migration.sql.
2. Применяет SQL к базе данных (создаёт таблицы Author и Book).
3. Генерирует повторно Prisma Client (обновляет типы и методы для новых моделей).

Вы увидите в консоли:

```
Applying migration `20240115_init`

The following migration(s) have been created and applied:
```

```
migrations/
  20240115103000_init/
    migration.sql
```

Откройте сгенерированный файл migration.sql и посмотрите, что Prisma создала:

```
-- CreateTable
CREATE TABLE "Author" (
    "id" SERIAL NOT NULL,
    "name" TEXT NOT NULL,
    "email" TEXT NOT NULL,
    "bio" TEXT,
    "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt" TIMESTAMP(3) NOT NULL,

    CONSTRAINT "Author_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "Book" (
    "id" SERIAL NOT NULL,
    "title" TEXT NOT NULL,
    "isbn" TEXT NOT NULL,
    "pages" INTEGER,
    "published" BOOLEAN NOT NULL DEFAULT false,
    "authorId" INTEGER NOT NULL,
    "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt" TIMESTAMP(3) NOT NULL,

    CONSTRAINT "Book_pkey" PRIMARY KEY ("id")
);

-- CreateIndex
CREATE UNIQUE INDEX "Author_email_key" ON "Author"("email");
CREATE UNIQUE INDEX "Book_isbn_key" ON "Book"("isbn");

-- AddForeignKey
ALTER TABLE "Book" ADD CONSTRAINT "Book_authorId_fkey"
    FOREIGN KEY ("authorId") REFERENCES "Author"("id") ON DELETE
RESTRICT;
```

Обратите внимание на FOREIGN KEY: PostgreSQL гарантирует, что каждая книга ссылается на существующего автора. Попытка создать книгу с несуществующим authorId вызовет ошибку базы данных.

Изменение схемы и новая миграция

Допустим, вы хотите добавить поле genre (жанр) к модели Book. Измените модель Book в schema.prisma:

```
model Book {
    id        Int      @id @default(autoincrement())
    title     String
    isbn      String   @unique
    pages     Int?
    genre     String?                // <-- новое поле
    published Boolean  @default(false)
    author    Author   @relation(fields: [authorId], references: [id])
    authorId  Int
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt
}
```

Создайте новую миграцию:

```
npx prisma migrate dev --name add_genre_to_book
```

Prisma создаст новый файл миграции и в нем следующий SQL запрос:

```
ALTER TABLE "Book" ADD COLUMN "genre" TEXT;
```

Найдите этот файл миграции и откройте его, чтобы проверить.

Полезные команды Prisma

```
npx prisma migrate dev --name <name> # Создать и применить миграцию
npx prisma migrate deploy            # Применить миграции (продакшен)
npx prisma migrate reset             # Сбросить БД и применить заново
npx prisma db push                   # Применить схему без миграции
(прототипирование)
npx prisma generate                  # Сгенерировать клиент
npx prisma studio                    # Визуальный редактор данных
```

Prisma Studio

Prisma Studio это веб-морда для просмотра и редактирования данных в базе. Запустите команду:

```
npx prisma studio
```

Откроется браузер на http://localhost:5555. Вы увидите все таблицы и сможете добавлять, редактировать и удалять записи через удобный интерфейс. Это полезно для отладки и проверки данных.

7.6 CRUD-операции

PrismaClient предоставляет типобезопасный API для всех операций. Он автоматически генерируется из вашей схемы и каждая модель становится свойством объекта prisma: prisma.author, prisma.book.

Создайте файл prisma/client.js для создания экземпляра клиента:

```
import { PrismaClient } from '@prisma/client';


const prisma = new PrismaClient();


export default prisma;
```

Важно: создавайте один экземпляр PrismaClient на всё приложение. Не создавайте новый экземпляр в каждом файле — это приведёт к исчерпанию пула подключений к базе данных.

Создание server.js

Создадим главный файл приложения server.js. В отличие от Mongoose, Prisma не требует явного вызова connect() — PrismaClient подключается к базе данных автоматически при первом запросе (lazy connection). Поэтому server.js будет проще:

```
import express from 'express';
import authorsRouter from './routes/authors.js';
import booksRouter from './routes/books.js';
const app = express();
const PORT = process.env.PORT || 3000;
app.use(express.json());

// Логирование запросов
app.use((req, res, next) => {
    console.log(`${new Date().toISOString()} ${req.method}
${req.url}`);
    next();
});
// Маршруты
app.use('/api/authors', authorsRouter);
app.use('/api/books', booksRouter);
// 404 — маршрут не найден
app.use((req, res) => {
    res.status(404).json({ error: 'Маршрут не найден' });
});
```

```
// Централизованный обработчик ошибок
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(err.status || 500).json({
        error: err.message || 'Внутренняя ошибка сервера',
    });
});
app.listen(PORT, () => {
    console.log(`Сервер: http://localhost:${PORT}`);
});
```

Обратите внимание: в server.js мы не импортируем файл prisma/client.js напрямую — он используется только внутри маршрутов. Так же для нашего приложения нужен будет express. Установим его:

```
npm install express
```

Добавляем CRUD-маршруты для авторов

Создайте файл routes/authors.js:

```
import { Router } from 'express';
import prisma from '../prisma/client.js';


const router = Router();
```

Create — создание автора

Метод prisma.author.create() создаёт запись в таблице Author. Поле data содержит значения полей:

```
router.post('/', async (req, res, next) => {
    try {
        const author = await prisma.author.create({
            data: req.body,
        });
        res.status(201).json(author);
    } catch (err) {
        if (err.code === 'P2002') {
            return res.status(409).json({ error: 'Email уже существует'
});
        }
        next(err);
    }
});
```

Код ошибки P2002 — уникальное ограничение нарушено (в нашем случае email). Prisma использует собственные коды ошибок вместо кодов БД.

Read — получение списка авторов с постраничным выводом

Метод prisma.author.findMany() возвращает массив записей. Поддерживает фильтрацию (where), сортировку (orderBy), пагинацию (skip/take) и подгрузку связей (include):

```
router.get('/', async (req, res, next) => {
    try {
        const { page = 1, limit = 10 } = req.query;
        const skip = (page - 1) * limit;

        const [authors, total] = await Promise.all([
            prisma.author.findMany({
                skip: Number(skip),
                take: Number(limit),
                orderBy: { createdAt: 'desc' },
                include: { books: true },
            }),
            prisma.author.count(),
        ]);

        res.json({
            authors,
            total,
            page: Number(page),
            pages: Math.ceil(total / limit),
        });
    } catch (err) { next(err); }
});
```

Сравнение с Mongoose:

- Mongoose: User.find().skip(20).limit(10).sort({ createdAt: -1 })

- Prisma: prisma.author.findMany({ skip: 20, take: 10, orderBy: { createdAt: 'desc' } })

В Prisma все параметры передаются как объект, а не через цепочку методов.

Read — получение автора по ID

Метод prisma.author.findUnique() ищет запись по уникальному полю (id или email). Возвращает null, если не найден:

```
router.get('/:id', async (req, res, next) => {
    try {
        const author = await prisma.author.findUnique({
            where: { id: Number(req.params.id) },
            include: { books: true },
        });
        if (!author) {
            return res.status(404).json({ error: 'Автор не найден' });
        }
        res.json(author);
    } catch (err) { next(err); }
});
```

Обратите внимание: req.params.id приходит как строка, а в Prisma поле id имеет тип Int. Поэтому нужно Number(req.params.id).

include: { books: true } подгружает все книги автора. Это аналог populate() в Mongoose, но реализуется через SQL JOIN.

Update — обновление автора

Метод prisma.author.update() находит запись по where и обновляет поля из data:

```
router.patch('/:id', async (req, res, next) => {
    try {
        const author = await prisma.author.update({
            where: { id: Number(req.params.id) },
            data: req.body,
        });
        res.json(author);
    } catch (err) {
        if (err.code === 'P2025') {
            return res.status(404).json({ error: 'Автор не найден' });
        }
        next(err);
    }
});
```

Код ошибки P2025 означает, что запись не найдена. В отличие от Mongoose (findByIdAndUpdate возвращает null), Prisma выбрасывает ошибку.

Delete — удаление автора

Метод prisma.author.delete() удаляет запись. Если у автора есть книги, PostgreSQL не позволит удаление из-за внешнего ключа (RESTRICT по умолчанию):

```
router.delete('/:id', async (req, res, next) => {
    try {
        await prisma.author.delete({
            where: { id: Number(req.params.id) },
        });
        res.status(204).end();
    } catch (err) {
        if (err.code === 'P2025') {
            return res.status(404).json({ error: 'Автор не найден' });
        }
        if (err.code === 'P2003') {
            return res.status(409).json({ error: 'Нельзя удалить автора
с книгами' });
        }
        next(err);
    }
});

export default router;
```

Код P2003: нарушение внешнего ключа. PostgreSQL не даст удалить автора, пока на него ссылаются книги. Это гарантия целостности данных, которой нет в MongoDB.

CRUD-маршруты для книг

Теперь добавим маршруты для книг. Создайте файл routes/books.js:

```
import { Router } from 'express';
import prisma from '../prisma/client.js';

const router = Router();
```

```
// CREATE
router.post('/', async (req, res, next) => {
    try {
        const book = await prisma.book.create({
            data: req.body,
            include: { author: true },
        });
        res.status(201).json(book);
    } catch (err) {
        if (err.code === 'P2002') {
            return res.status(409).json({ error: 'ISBN уже существует'
});
        }
        if (err.code === 'P2003') {
            return res.status(400).json({ error: 'Автор не найден' });
        }
        next(err);
    }
});

// READ ALL
router.get('/', async (req, res, next) => {
    try {
        const { page = 1, limit = 10, published } = req.query;
        const where = published !== undefined
            ? { published: published === 'true' }
            : {};

        const [books, total] = await Promise.all([
            prisma.book.findMany({
                where,
                skip: (page - 1) * limit,
                take: Number(limit),
                orderBy: { createdAt: 'desc' },
                include: {
                    author: { select: { id: true, name: true, email:
true } },
                },
            }),
            prisma.book.count({ where }),
        ]);

        res.json({ books, total, page: Number(page), pages:
Math.ceil(total / limit) });
    } catch (err) { next(err); }
});

// READ ONE
```

```
router.get('/:id', async (req, res, next) => {
    try {
        const book = await prisma.book.findUnique({
            where: { id: Number(req.params.id) },
            include: { author: true },
        });
        if (!book) return res.status(404).json({ error: 'Книга не
найдена' });
        res.json(book);
    } catch (err) { next(err); }
});

// UPDATE
router.patch('/:id', async (req, res, next) => {
    try {
        const book = await prisma.book.update({
            where: { id: Number(req.params.id) },
            data: req.body,
            include: { author: true },
        });
        res.json(book);
    } catch (err) {
        if (err.code === 'P2025') return res.status(404).json({ error:
'Книга не найдена' });
        next(err);
    }
});

// DELETE
router.delete('/:id', async (req, res, next) => {
    try {
        await prisma.book.delete({ where: { id: Number(req.params.id) }
});
        res.status(204).end();
    } catch (err) {
        if (err.code === 'P2025') return res.status(404).json({ error:
'Книга не найдена' });
        next(err);
    }
});

export default router;
```

Обратите внимание на include и select в маршрутах post, get, patch:

- include: { author: true } подгружает все поля автора (аналог populate в Mongoose).
- include: { author: { select: { id: true, name: true } } } подгружает только указанные поля автора.

В server.js мы уже подключали маршруты, когда только создали его

```
// Маршруты
app.use('/api/authors', authorsRouter);
app.use('/api/books', booksRouter);
```

Все маршруты для авторов будут доступны по префиксу /api/authors:

- POST /api/authors — создать автора

- GET /api/authors — получить список

- GET /api/authors/:id — получить по ID

- PATCH /api/authors/:id — обновить

- DELETE /api/authors/:id — удалить

Все маршруты для книг доступны по префиксу /api/books:

- POST /api/books — создать автора

- GET /api/books — получить список

- GET /api/books/:id — получить по ID

- PATCH /api/books/:id — обновить

- DELETE /api/books/:id — удалить

Запустим сервер и проверим все маршруты:

Создание автора

```
curl -X POST http://localhost:3000/api/authors \
  -H 'Content-Type: application/json' \
  -d '{"name": "Лев Толстой", "email": "tolstoy@example.com", "bio":
"Русский писатель"}'
```

Ожидаемый ответ (статус 201):

```
{
    "id": 1,
    "name": "Лев Толстой",
    "email": "tolstoy@example.com",
    "bio": "Русский писатель",
    "createdAt": "2024-01-15T10:30:00.000Z",
    "updatedAt": "2024-01-15T10:30:00.000Z"
}
```

На Windows может возникать проблема, что json не парсится. Поэтому нужно обернуть payload всех POST/PUT/PATCH запросов, а также Content-Type: application/json в двойные кавычки вместо одиночных. Пример:

```
curl -X POST http://localhost:3000/api/authors \
  -H "Content-Type: application/json" \
  -d "{\"name\": \"Лев Толстой\", \"email\": \"tolstoy@example.com\",
\"bio\": \"Русский писатель\"}"
```

Чтение всех авторов

```
curl http://localhost:3000/api/authors
curl "http://localhost:3000/api/authors?page=1&limit=5"
```

Чтение одного автора по ID

Подставьте реальный _id из ответа на создание или чтение всех вместо <_id> :

```
curl http://localhost:3000/api/authors/<_id>
```

Можете подставить заведомо ложный id автора, чтобы вывести ошибку

```
curl http://localhost:3000/api/authors/12345
```

API вернет

```
{"error":"Автор не найден"}
```

Обновление автора

Подставьте реальный id вместо <_id> :

```
curl -X PATCH http://localhost:3000/api/authors/<_id> \
  -H 'Content-Type: application/json' \
  -d '{"bio": "Великий русский писатель"}'
```

В ответе вернётся обновлённый пользователь с новым именем и обновлённым полем updatedAt.

Удаление автора

Подставьте реальный id вместо <_id> :

```
curl -X DELETE http://localhost:3000/api/authors/<_id>
```

Ответ: пустое тело, статус 204. Повторный запрос на тот же ID вернёт 404.

Создание книги:

```
curl -X POST http://localhost:3000/api/books \
  -H 'Content-Type: application/json' \
  -d '{"title": "Война и мир", "isbn": "978-5-17-006309-7", "pages":
1225, "authorId": 1}'
```

Обратите внимание мы передаем authorId=1 (ID созданного автора) если передать другую цифру, то будет ошибка. Так же обратите внимание поле isbn должно быть уникальным в таблице, соответственно мы не сможем добавить несколько книг с одинаковым isbn.

Остальные запросы на выборку, изменение и удаление работают аналогично как у авторов, поэтому можно просто проверить следующими запросами

```
curl http://localhost:3000/api/books

curl http://localhost:3000/api/books/<_id>

curl -X PATCH http://localhost:3000/api/books/<_id> \
  -H 'Content-Type: application/json' \
  -d '{"title": "Новая книга"}'

curl -X DELETE http://localhost:3000/api/books/<_id>
```

Коды ошибок Prisma

Основные коды, которые нужно обрабатывать в маршрутах:

- P2002: нарушение уникального ограничения (duplicate).
- P2003: нарушение внешнего ключа (foreign key constraint failed).
- P2025: запись не найдена (record not found).

7.7 Фильтрация и транзакции

Prisma поддерживает сложные фильтры с логическими операторами AND, OR, NOT и операторами сравнения contains, startsWith, gt, lt и другими.

Транзакции гарантируют, что несколько операций выполнятся атомарно — либо все, либо ни одна.

Фильтрация в Prisma

Prisma поддерживает мощную систему фильтрации через объект where. В отличие от MongoDB, где фильтры пишутся в стиле { field: { $gt: 10 } }, Prisma использует вложенные объекты с именованными операторами.

Пример операторов сравнения

```
// Точное совпадение
const book = await prisma.book.findMany({
    where: { published: true },
});

// contains — поиск подстроки (аналог $regex в MongoDB)
const books = await prisma.book.findMany({
    where: {
        title: { contains: 'война', mode: 'insensitive' },
    },
});



// startsWith, endsWith
const books2 = await prisma.book.findMany({
    where: { title: { startsWith: 'Война' } },
```

```
});

// gt, gte, lt, lte — сравнение чисел и дат
const longBooks = await prisma.book.findMany({
    where: { pages: { gt: 500 } },
});

// in — значение входит в список
const selected = await prisma.book.findMany({
    where: { id: { in: [1, 2, 3] } },
});
```

Параметр mode: 'insensitive' делает поиск регистронезависимым. Это работает для PostgreSQL, но не для всех баз данных.

Пример логических операторов AND, OR, NOT

```
// AND — все условия должны выполняться
const books = await prisma.book.findMany({
    where: {
        AND: [
            { published: true },
            { pages: { gt: 300 } },
        ],
    },
});

// OR — хотя бы одно условие
const books2 = await prisma.book.findMany({
    where: {
        OR: [
            { title: { contains: 'война' } },
            { title: { contains: 'мир' } },
        ],
    },
});

// NOT — исключение
const unpublished = await prisma.book.findMany({
    where: { NOT: { published: true } },
});
```

Фильтрация по связанным моделям

Prisma так же позволяет фильтровать записи по полям связанных моделей. Это мощная возможность, аналога которой нет в базовом Mongoose:

```
// Найти все книги, автор которых имеет email 'tolstoy@example.com'
const books = await prisma.book.findMany({
    where: {
        author: { email: 'tolstoy@example.com' },
    },
    include: { author: true },
});

// Найти авторов, у которых есть хотя бы одна опубликованная книга
const authors = await prisma.author.findMany({
    where: {
        books: { some: { published: true } },
    },
});
```

Операторы для связей:

- some: хотя бы одна связанная запись удовлетворяет условию.
- every: все связанные записи удовлетворяют условию.
- none: ни одна связанная запись не удовлетворяет условию.

Добавим поиск книг

Добавьте в routes/books.js маршрут поиска (**обязательно** перед маршрутом /:id, так как иначе роутер будет думать, что вы передаете search в качестве id и выдавать ошибку, что такой книги нет!):

```
router.get('/search', async (req, res, next) => {
    try {
        const { q, published, minPages } = req.query;
        if (!q) return res.status(400).json({ error: 'Параметр q
обязателен' });

        const where = {
            title: { contains: q, mode: 'insensitive' },
        };
```

```
        if (published !== undefined) where.published = published ===
'true';
        if (minPages) where.pages = { gte: Number(minPages) };

        const books = await prisma.book.findMany({
            where,
            include: { author: { select: { name: true } } },
            orderBy: { title: 'asc' },
        });
        res.json(books);
    } catch (err) { next(err); }
});
```

Проверка:

```
curl "http://localhost:3000/api/books/search?q=война"
```

Обратите внимание: при выполнении этой команды из командной строки скорее всего вернется пустой массив, так как CMD не кодирует кириллицу и может передать строку в кодировке Windows-1251 вместо UTF-8, что даст некорректный результат. Поэтому для решения проблемы можно выполнить следующую команду и должно все работать. В реальных приложениях и запросах с фронтенда этого не потребуется, так как там все обычно уж в кодировке UTF-8.

```
curl -G http://localhost:3000/api/books/search --data-urlencode
"q=война"
```

Транзакции

Транзакция гарантирует, что несколько операций выполнятся атомарно: либо все успешно, либо все откатятся. Это критически важно для целостности данных, например в финтех приложениях (банковские транзакции, платежные системы и т.п.).

Пример: создание автора и его книги в одной транзакции:

```
const [author, book] = await prisma.$transaction([
    prisma.author.create({
        data: { name: 'Достоевский', email: 'dostoevsky@example.com' },
    }),
    prisma.book.create({
        data: {
            title: 'Преступление и наказание',
            isbn: '978-5-17-090295-0',
```

```
            pages: 672,
            authorId: 2,
        },
    }),
]);
```

Если создание книги упадёт (например, ISBN не уникален), создание автора тоже откатится.

Транзакции бывают интерактивными

Для более сложных сценариев, где результат одной операции нужен для следующей, используйте интерактивную транзакцию. Пример:

```
const result = await prisma.$transaction(async (tx) => {
    // Шаг 1: создаём автора
    const author = await tx.author.create({
        data: { name: 'Чехов', email: 'chekhov@example.com' },
    });

    // Шаг 2: создаём книгу, используя id автора из шага 1
    const book = await tx.book.create({
        data: {
            title: 'Вишнёвый сад',
            isbn: '978-5-17-090300-1',
            authorId: author.id,
        },
    });

    return { author, book };
});
```

В интерактивной транзакции используется параметр tx вместо prisma. Все операции внутри callback выполняются в одной транзакции. Если любая операция вызовет ошибку или вы выбросите throw, все изменения откатятся.

Вложенное создание (nested create)

Prisma позволяет создавать связанные записи в одном запросе без явной транзакции. Пример:

```
const author = await prisma.author.create({
    data: {
```

```
        name: 'Пушкин',
        email: 'pushkin@example.com',
        books: {
            create: [
                { title: 'Евгений Онегин', isbn: '978-5-04-001001-1',
pages: 448 },
                { title: 'Капитанская дочка', isbn: '978-5-04-001002-
8', pages: 320 },
            ],
        },
    },
    include: { books: true },
});
```

Prisma создаст автора и две книги в одной транзакции, автоматически подставив authorId. Это удобнее и безопаснее, чем создавать записи по отдельности.

Итак, попробуем добавлять записи в базу с помощью транзакций

Чтобы ломать уже созданные маршруты, давайте создадим новый маршрут для вставки данных транзакциями

Создайте файл routes/transactions.js:

```
import { Router } from 'express';
import prisma from '../prisma/client.js';

const router = Router();

// Демо 1: Последовательная транзакция — создание автора + книги
router.post('/sequential', async (req, res, next) => {
    try {
        const { authorName, authorEmail, bookTitle, bookIsbn, bookPages
} = req.body;
        const [author, book] = await prisma.$transaction([
            prisma.author.create({
                data: { name: authorName, email: authorEmail },
            }),
            prisma.book.create({
                data: {
                    title: bookTitle,
                    isbn: bookIsbn,
                    pages: bookPages || null,
                    authorId: 0, // будет заменён в интерактивной
транзакции
                },
            }),
```

```
        ]);

        res.status(201).json({ message: 'Транзакция успешна', author,
book });
    } catch (err) {
        res.status(400).json({
            message: 'Транзакция откатилась — ни автор, ни книга НЕ
созданы',
            error: err.message,
        });
    }
});

// Демо 2: Интерактивная транзакция — создание автора, затем книги с
его id
router.post('/interactive', async (req, res, next) => {
    try {
        const { authorName, authorEmail, bookTitle, bookIsbn, bookPages
} = req.body;

        const result = await prisma.$transaction(async (tx) => {
            // Шаг 1: создаём автора
            const author = await tx.author.create({
                data: { name: authorName, email: authorEmail },
            });

            // Шаг 2: создаём книгу, используя id автора из шага 1
            const book = await tx.book.create({
                data: {
                    title: bookTitle,
                    isbn: bookIsbn,
                    pages: bookPages || null,
                    authorId: author.id,
                },
            });

            return { author, book };
        });

        res.status(201).json({ message: 'Транзакция успешна', ...result
});
    } catch (err) {
        res.status(400).json({
            message: 'Транзакция откатилась — ни автор, ни книга НЕ
созданы',
            error: err.message,
        });
    }
});
```

```
// Вспомогательный: проверить текущее состояние БД
router.get('/check', async (req, res, next) => {
    try {
        const authors = await prisma.author.findMany({
            orderBy: { id: 'desc' },
            take: 5,
            include: { books: { select: { id: true, title: true, isbn:
true } } },
        });
        const totalAuthors = await prisma.author.count();
        const totalBooks = await prisma.book.count();
        res.json({ totalAuthors, totalBooks, lastAuthors: authors });
    } catch (err) { next(err); }
});

export default router;
```

и подключим его в server.js

```
import transactionsRouter from './routes/transactions.js';

// Вставим после app.use('/api/books', booksRouter);
app.use('/api/transactions', transactionsRouter);
```

Прежде чем начать, посмотрим, сколько авторов и книг сейчас в базе. Выполним запрос:

```
curl http://localhost:3000/api/transactions/check
```

Пример ответа:

```
{
    "totalAuthors": 1,
    "totalBooks": 1,
    "lastAuthors": [
        {
            "id": 1,
            "name": "Лев Толстой",
            "email": "tolstoy@example.com",
            "bio": null,
            "createdAt": "2026-03-18T15:00:53.434Z",
            "updatedAt": "2026-03-18T15:00:53.434Z",
            "books": [
                { "id": 1, "title": "Война", "isbn": "978-5-17-006309-
7" }
            ]
        }
```

```
    ]
}
```

Запомним: в базе 1 автор и 1 книга.

Успешная транзакция (интерактивная)

Отправим корректные данные — создадим нового автора и его книгу в одной транзакции. Используется интерактивная транзакция: сначала создаётся автор, затем его id подставляется при создании книги.

```
curl -X POST http://localhost:3000/api/transactions/interactive \
  -H "Content-Type: application/json" \
  -d '{"authorName": "Фёдор Достоевский", "authorEmail":
"dostoevsky@example.com", "bookTitle": "Преступление и наказание",
"bookIsbn": "978-5-17-090295-0", "bookPages": 672}'
```

Ожидаемый ответ (статус 201):

```
curl -s -X POST http://localhost:3000/api/transactions/interactive \
  -{
    "message": "Транзакция успешна",
    "author": {
        "id": 2,
        "name": "Фёдор Достоевский",
        "email": "dostoevsky@example.com",
        "bio": null,
        "createdAt": "...",
        "updatedAt": "..."
    },
    "book": {
        "id": 2,
        "title": "Преступление и наказание",
        "isbn": "978-5-17-090295-0",
        "pages": 672,
        "genre": null,
        "published": false,
        "authorId": 2,
        "createdAt": "...",
        "updatedAt": "..."
    }
}
```

Транзакция прошла успешно. Обратите внимание: authorId книги равен id только что созданного автора. Это стало возможным благодаря интерактивной транзакции — мы получили id автора на шаге 1 и использовали его на шаге 2. Смотрите файл routes/transactions.js

Убедимся, что данные сохранились:

```
curl http://localhost:3000/api/transactions/check
```

Результат: totalAuthors = 2, totalBooks = 2. Оба объекта созданы.

Неудачная транзакция — откат

Теперь отправим заведомо некорректные данные. Попробуем создать нового автора и книгу, но укажем ISBN, который уже существует в базе (978-5-17-090295-0 — мы только что его добавили). Поле isbn имеет ограничение @unique, поэтому создание книги завершится ошибкой.

```
curl -X POST http://localhost:3000/api/transactions/interactive \
  -H 'Content-Type: application/json' \
  -d '{"authorName": "Антон Чехов", "authorEmail":
"chekhov@example.com", "bookTitle": "Вишнёвый сад", "bookIsbn": "978-5-
17-090295-0", "bookPages": 256}'
```

Ожидаемый ответ (статус 400):

```
{
    "message": "Транзакция откатилась — ни автор, ни книга НЕ созданы",
    "error": "Unique constraint failed on the fields: (`isbn`)"
}
```

Создание книги не удалось из-за дублирования ISBN. Благодаря транзакции создание автора «Антон Чехов» тоже откатилось. Убедимся, что база не изменилась:

```
curl http://localhost:3000/api/transactions/check
```

Результат: totalAuthors = 2, totalBooks = 2. Ничего не добавилось — автор «Чехов» НЕ был создан, несмотря на то, что операция его создания стояла первой и сама по себе была корректной. Транзакция откатила всё целиком.

Если бы мы выполняли две операции отдельно (без транзакции), произошло бы следующее:

1. Автор «Чехов» создался бы успешно (email уникален).

2. Книга с дублирующим ISBN — ошибка.
3. В итоге: автор есть, а книги нет. База в неконсистентном состоянии.

Транзакция защищает от таких ситуаций.

7.8 Переменные окружения

Переменные окружения — это специальные именованные значения, которые хранятся вне программы и передаются ей из среды выполнения. Они используются для настройки поведения приложения без изменения его кода.

По сути, это способ сообщить программе, в каких условиях она работает и какие параметры ей нужно использовать. Например, с их помощью можно указать настройки подключения, режим работы или другие конфигурационные данные.

Главное преимущество переменных окружения в том, что они отделяют конфигурацию от логики программы. Это позволяет использовать один и тот же код в разных средах (разработка, тестирование, продакшн), меняя только значения переменных.

Также они помогают повысить безопасность, поскольку чувствительные данные не хранятся напрямую в коде, а передаются извне.

В целом, переменные окружения — это универсальный механизм управления настройками приложений, который делает разработку более гибкой, безопасной и удобной.

Prisma использует переменные окружения env("DATABASE_URL") и в schema.prisma читает значение из файла .env.

Но в нашем приложении могут быть и другие переменные: порт сервера, секреты для JWT и т.д.

Файл .env

Откройте файл .env в корне проекта:

```
DATABASE_URL="postgresql://pg_user:pg_pass@localhost:5432/myapp"
```

На данный момент файл содержит одну настройку. Это путь для подключения к базе данных PostgreSQL.
Давайте изменим файл и добавим PORT и NODE_ENV

```
DATABASE_URL="postgresql://pg_user:pg_pass@localhost:5432/myapp"
PORT=3010 # <-  обратите внимание на порт, мы изменили его на 3010
NODE_ENV=development
```

Помните в начале модуля мы добавляли скрипты в package.json? И ваш package.json должен выглядеть как-то так:

```
{
  "name": "module07",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node --env-file=.env server.js",
    "dev": "node --env-file=.env --watch server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "type": "module",
  "devDependencies": {
    "prisma": "^6.19.2"
  },
  "dependencies": {
    "@prisma/client": "^6.19.2",
    "express": "^5.2.1"
  }
}
```

Скрипты запускает сервер и подключает файл .env при запуске.

```
    "start": "node --env-file=.env server.js",
    "dev": "node --env-file=.env --watch server.js"
```

Соответственно все переменные из этого файла будут доступные внутри приложения через переменную process.env

Давайте попробуем добавить еще две переменных окружения в файл .env

```
MY_CUSTOM_VAR="my custom variable"
ANOTHER_CUSTOM_VAR=AnotherCustomVar
```

Обратите внимание что значения с пробелами, заключаются в двойные кавычки.

Теперь изменим файл server.js и выведем переменные в логи

```
import express from 'express';
import authorsRouter from './routes/authors.js';
import booksRouter from './routes/books.js';
import transactionsRouter from './routes/transactions.js';

const app = express();
const PORT = process.env.PORT || 3000;

console.log('Переменная окружения MY_CUSTOM_VAR:',
process.env.MY_CUSTOM_VAR);
console.log('Переменная окружения ANOTHER_CUSTOM_VAR:',
process.env.ANOTHER_CUSTOM_VAR);

app.use(express.json());
// Логирование запросов
app.use((req, res, next) => {
    console.log(`${new Date().toISOString()} ${req.method}
${req.url}`);
    next();
});
// Маршруты
app.use('/api/authors', authorsRouter);
app.use('/api/books', booksRouter);
app.use('/api/transactions', transactionsRouter);
// 404 — маршрут не найден
app.use((req, res) => {
    res.status(404).json({ error: 'Маршрут не найден' });
});
// Централизованный обработчик ошибок
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(err.status || 500).json({
        error: err.message || 'Внутренняя ошибка сервера',
    });
});
app.listen(PORT, () => {
    console.log(`Сервер: http://localhost:${PORT}`);
});
```

Запустим приложение с помощью, выполнив:

```
npm run dev
```

В консоли вы увидите, что-то вроде этого

```
Переменная окружения MY_CUSTOM_VAR: my custom variable
Переменная окружения ANOTHER_CUSTOM_VAR: AnotherCustomVar
Сервер: http://localhost:3010
Completed running 'server.js'
```

Значения наших переменных были переданы в приложение. Обратите внимание на порт, теперь он 3010, а не 3000, так как мы его изменили.

Файлы .env.example и .gitignore

Обычно в git репозиторий файл .env никогда не коммитится, так как он может содержать какие-то секретные данные, типа паролей, ключей и так далее. Поэтому для того, чтобы передать другим разработчикам шаблон .env, создается другой файл .env.example, который содержит названия переменных без значений и коммитится в git.

Создайте .env.example (шаблон для репозитория):

```
PORT=3010
DATABASE_URL=
NODE_ENV=development
MY_CUSTOM_VAR=
ANOTHER_CUSTOM_VAR=
```

Откройте или создайте .gitignore в корне проекта (точка перед именем обязательна).

.gitignore — это специальный файл в системе контроля версий Git, который определяет, какие файлы и папки не должны отслеживаться и добавляться в репозиторий.

Он используется для того, чтобы исключить из проекта лишние или временные данные: служебные файлы, результаты сборки, локальные настройки, зависимости и другие элементы, которые не относятся к исходному коду или не должны попадать в общий доступ.

Файл .gitignore помогает поддерживать репозиторий в чистоте, уменьшает его размер и предотвращает случайную публикацию чувствительной или ненужной информации.

В целом, .gitignore — это инструмент управления содержимым репозитория, который позволяет разработчикам контролировать, какие файлы важны для проекта, а какие следует игнорировать.

Содержимое файла .gitignore должно быть таким:

```
node_modules
.env
/generated/prisma
```

Папка node_modules не нужна нам в git, так как она устанавливается с помощью npm install исходя из зависимостей в package.json

.env тоже не нужен, так как он может содержать разные значения в зависимости от окружения, где будет работать приложение.

/generated/prisma сгенерированные рантайм-файлы схемы тоже не нужны так как они полностью зависят от файла схемы /prisma/schema.prisma

Создадим git репозиторий и закоммитим наше приложение:

```
git init
git add .
git commit -m "My Express App"
```

И проверим какие файлы у нас в репозитории

```
git show --name-only
```

Файлы

```
.env.example
.gitignore
package-lock.json
package.json
prisma.config.ts
prisma/client.js
prisma/migrations/20260322145018_init/migration.sql
prisma/migrations/20260322145224_add_genre_to_book/migration.sql
prisma/migrations/migration_lock.toml
prisma/schema.prisma
routes/authors.js
routes/books.js
routes/transactions.js
server.js
```

7.9 Express + Prisma: полноценное приложение

В этом разделе мы соберём всё, что изучили в модуле, в одно полноценное приложение. Пройдём каждый шаг заново — от создания проекта до тестирования всех эндпоинтов. Если вы внимательно прошли разделы 7.1–7.8, код будет вам знаком.

Структура проекта

```
my-prisma-app/
├── prisma/
│   ├── schema.prisma       # Схема моделей
│   ├── client.js           # Экземпляр PrismaClient
│   └── migrations/         # Файлы миграций
├── routes/
│   ├── authors.js          # CRUD авторов
│   └── books.js            # CRUD книг + поиск
├── server.js               # Главный файл
├── .env                    # Переменные окружения
├── .env.example            # Шаблон (для репозитория)
├── .gitignore
└── package.json
```

Шаг 1. Инициализация проекта

Создайте папку проекта и инициализируйте его:

```
mkdir my-prisma-app
cd my-prisma-app
npm init -y
```

Включите ESM-модули — откройте package.json и добавьте:

```
"type": "module"
```

Установите зависимости:

```
npm install express @prisma/client
npm install prisma --save-dev
```

Инициализируйте Prisma:

```
npx prisma init
```

Эта команда создаст папку prisma/ с файлом schema.prisma и файл .env с переменной DATABASE_URL.

Шаг 2. package.json

Убедитесь, что ваш package.json содержит скрипты с флагом --env-file=.env (как мы настраивали в разделе 7.8):

```
{
    "name": "my-prisma-app",
    "type": "module",
    "scripts": {
        "start": "node --env-file=.env server.js",
        "dev": "node --env-file=.env --watch server.js",
        "migrate": "npx prisma migrate dev",
        "studio": "npx prisma studio"
    },
    "dependencies": {
        "@prisma/client": "^6.x",
        "express": "^5.x"
    },
    "devDependencies": {
        "prisma": "^6.x"
    }
}
```

Скрипт dev использует --watch (Node.js 18+) для автоперезагрузки и --env-file=.env для загрузки переменных окружения без сторонних пакетов.

Шаг 3. Файл .env

Откройте файл .env в корне проекта и настройте переменные:

```
DATABASE_URL="postgresql://pguser:pgpass@localhost:5432/myapp"
PORT=3000
NODE_ENV=development
```

Если вы используете Docker (раздел 7.2), убедитесь, что контейнер postgres_db запущен:

```
docker start postgres_db
```

Шаг 4. Файл .env.example

Создайте шаблон для репозитория (без реальных паролей):

```
DATABASE_URL="postgresql://USER:PASSWORD@HOST:PORT/DB_NAME"
PORT=3000
NODE_ENV=development
```

Шаг 5. Файл .gitignore

```
node_modules
.env
/generated/prisma
```

Шаг 6. prisma/schema.prisma

Опишите модели Author и Book (как в разделе 7.4):

```
generator client {
    provider = "prisma-client-js"
}

datasource db {
    provider = "postgresql"
    url      = env("DATABASE_URL")
}

model Author {
    id        Int      @id @default(autoincrement())
    name      String
    email     String   @unique
    bio       String?
    books     Book[]
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt
}

model Book {
    id        Int      @id @default(autoincrement())
    title     String
    isbn      String   @unique
    pages     Int?
    published Boolean  @default(false)
    author    Author   @relation(fields: [authorId], references: [id])
    authorId  Int
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt
}
```

Шаг 7. Создание миграции

Выполните первую миграцию (как в разделе 7.5):

```
npx prisma migrate dev --name init
```

Prisma создаст таблицы Author и Book в базе данных и сгенерирует клиент.

Шаг 8. prisma/client.js

Создайте файл prisma/client.js — единственный экземпляр PrismaClient на всё приложение:

```
import { PrismaClient } from '@prisma/client';


const prisma = new PrismaClient();


export default prisma;
```

Шаг 9. routes/authors.js

Создайте файл routes/authors.js с полным CRUD для авторов:

```
import { Router } from 'express';
import prisma from '../prisma/client.js';


const router = Router();

// CREATE — создание автора
router.post('/', async (req, res, next) => {
    try {
        const author = await prisma.author.create({ data: req.body });
        res.status(201).json(author);
    } catch (err) {
        if (err.code === 'P2002') return res.status(409).json({ error:
'Email уже существует' });
        next(err);
    }
});

// READ ALL — список авторов с пагинацией
router.get('/', async (req, res, next) => {
    try {
        const { page = 1, limit = 10 } = req.query;
```

```
        const [authors, total] = await Promise.all([
            prisma.author.findMany({
                skip: (page - 1) * limit,
                take: Number(limit),
                orderBy: { createdAt: 'desc' },
                include: { books: true },
            }),
            prisma.author.count(),
        ]);
        res.json({ authors, total, page: Number(page), pages:
Math.ceil(total / limit) });
    } catch (err) { next(err); }
});

// READ ONE — получение автора по ID
router.get('/:id', async (req, res, next) => {
    try {
        const author = await prisma.author.findUnique({
            where: { id: Number(req.params.id) },
            include: { books: true },
        });
        if (!author) return res.status(404).json({ error: 'Автор не
найден' });
        res.json(author);
    } catch (err) { next(err); }
});

// UPDATE — обновление автора
router.patch('/:id', async (req, res, next) => {
    try {
        const author = await prisma.author.update({
            where: { id: Number(req.params.id) },
            data: req.body,
        });
        res.json(author);
    } catch (err) {
        if (err.code === 'P2025') return res.status(404).json({ error:
'Автор не найден' });
        next(err);
    }
});

// DELETE — удаление автора
router.delete('/:id', async (req, res, next) => {
    try {
        await prisma.author.delete({ where: { id: Number(req.params.id)
} });
        res.status(204).end();
    } catch (err) {
        if (err.code === 'P2025') return res.status(404).json({ error:
'Автор не найден' });
```

```
        if (err.code === 'P2003') return res.status(409).json({ error:
'Нельзя удалить автора с книгами' });
        next(err);
    }
});

export default router;
```

Шаг 10. routes/books.js

Создайте файл routes/books.js. Помимо стандартного CRUD, добавим маршрут поиска (фильтрация из раздела 7.7):

```
import { Router } from 'express';
import prisma from '../prisma/client.js';


const router = Router();

// CREATE — создание книги
router.post('/', async (req, res, next) => {
    try {
        const book = await prisma.book.create({
            data: req.body,
            include: { author: true },
        });
        res.status(201).json(book);
    } catch (err) {
        if (err.code === 'P2002') return res.status(409).json({ error:
'ISBN уже существует' });
        if (err.code === 'P2003') return res.status(400).json({ error:
'Автор не найден' });
        next(err);
    }
});

// SEARCH — поиск книг по названию (раздел 7.7)
// Важно: этот маршрут должен быть ДО /:id, иначе 'search' будет
воспринят как id
router.get('/search', async (req, res, next) => {
    try {
        const { q } = req.query;
        if (!q) return res.status(400).json({ error: 'Параметр q
обязателен' });
        const books = await prisma.book.findMany({
            where: { title: { contains: q, mode: 'insensitive' } },
            include: { author: { select: { id: true, name: true } } },
        });
        res.json(books);
    } catch (err) { next(err); }
```

```
});

// READ ALL — список книг с пагинацией и фильтрацией
router.get('/', async (req, res, next) => {
    try {
        const { page = 1, limit = 10, published } = req.query;
        const where = published !== undefined ? { published: published
=== 'true' } : {};
        const [books, total] = await Promise.all([
            prisma.book.findMany({
                where,
                skip: (page - 1) * limit,
                take: Number(limit),
                orderBy: { createdAt: 'desc' },
                include: { author: { select: { id: true, name: true,
email: true } } },
            }),
            prisma.book.count({ where }),
        ]);
        res.json({ books, total, page: Number(page), pages:
Math.ceil(total / limit) });
    } catch (err) { next(err); }
});

// READ ONE — получение книги по ID
router.get('/:id', async (req, res, next) => {
    try {
        const book = await prisma.book.findUnique({
            where: { id: Number(req.params.id) },
            include: { author: true },
        });
        if (!book) return res.status(404).json({ error: 'Книга не
найдена' });
        res.json(book);
    } catch (err) { next(err); }
});

// UPDATE — обновление книги
router.patch('/:id', async (req, res, next) => {
    try {
        const book = await prisma.book.update({
            where: { id: Number(req.params.id) },
            data: req.body,
            include: { author: true },
        });
        res.json(book);
    } catch (err) {
        if (err.code === 'P2025') return res.status(404).json({ error:
'Книга не найдена' });
        next(err);
    }
```

```
});

// DELETE — удаление книги
router.delete('/:id', async (req, res, next) => {
    try {
        await prisma.book.delete({ where: { id: Number(req.params.id) }
});
        res.status(204).end();
    } catch (err) {
        if (err.code === 'P2025') return res.status(404).json({ error:
'Книга не найдена' });
        next(err);
    }
});

export default router;
```

Шаг 11. server.js

Создайте главный файл server.js. Обратите внимание: мы не используем пакет dotenv — переменные окружения загружаются через флаг --env-file=.env в скриптах package.json (раздел 7.8). Prisma не требует явного connect() — подключение к БД происходит автоматически при первом запросе (lazy connection):

```
import express from 'express';
import authorsRouter from './routes/authors.js';
import booksRouter from './routes/books.js';

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

// Логирование запросов
app.use((req, res, next) => {
    console.log(`${new Date().toISOString()} ${req.method}
${req.url}`);
    next();
});

// Маршруты
app.use('/api/authors', authorsRouter);
app.use('/api/books', booksRouter);

// 404 — маршрут не найден
app.use((req, res) => {
    res.status(404).json({ error: 'Маршрут не найден' });
});
```

```
// Централизованный обработчик ошибок
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(err.status || 500).json({
        error: err.message || 'Внутренняя ошибка сервера',
    });
});

app.listen(PORT, () => {
    console.log(`Сервер: http://localhost:${PORT}`);
});
```

Шаг 12. Запуск

Убедитесь, что PostgreSQL запущен (Docker или локально), затем выполните:

```
npx prisma migrate dev --name init
npm run dev
```

В консоли вы увидите:

```
Сервер: http://localhost:3000
```

Шаг 13. Тестирование эндпоинтов

Проверим все маршруты с помощью curl. На Windows используйте двойные кавычки для JSON-данных (см. раздел 7.6).

Создание автора:

```
curl -X POST http://localhost:3000/api/authors \
  -H 'Content-Type: application/json' \
  -d '{"name": "Лев Толстой", "email": "tolstoy@example.com", "bio":
"Русский писатель"}'
```

Ожидаемый ответ (статус 201):

```
{
    "id": 1,
    "name": "Лев Толстой",
    "email": "tolstoy@example.com",
    "bio": "Русский писатель",
    "createdAt": "2024-01-15T10:30:00.000Z",
    "updatedAt": "2024-01-15T10:30:00.000Z"
}
```

Создание второго автора:

```
curl -X POST http://localhost:3000/api/authors \
  -H 'Content-Type: application/json' \
  -d '{"name": "Фёдор Достоевский", "email": "dostoevsky@example.com"}'
```

Создание книги (укажите authorId из ответа):

```
curl -X POST http://localhost:3000/api/books \
  -H 'Content-Type: application/json' \
  -d '{"title": "Война и мир", "isbn": "978-5-17-006309-7", "pages":
1225, "authorId": 1}'
```

В ответе будет книга с вложенным объектом автора (include: { author: true }).

Создание ещё одной книги:

```
curl -X POST http://localhost:3000/api/books \
  -H 'Content-Type: application/json' \
  -d '{"title": "Анна Каренина", "isbn": "978-5-17-006310-3", "pages":
864, "authorId": 1}'
```

Получение списка авторов с книгами:

```
curl http://localhost:3000/api/authors
```

В ответе каждый автор содержит массив books.

Получение автора по ID:

```
curl http://localhost:3000/api/authors/1
```

Обновление автора:

```
curl -X PATCH http://localhost:3000/api/authors/1 \
  -H 'Content-Type: application/json' \
  -d '{"bio": "Великий русский писатель"}'
```

Получение книг (с фильтрацией по published):

```
curl "http://localhost:3000/api/books?published=true"
```

Поиск книг по названию:

```
curl "http://localhost:3000/api/books/search?q=война"
```

Вернёт все книги, в названии которых есть «война» (регистронезависимый поиск).

Получение книги по ID:

```
curl http://localhost:3000/api/books/1
```

Обновление книги:

```
curl -X PATCH http://localhost:3000/api/books/1 \
  -H 'Content-Type: application/json' \
  -d '{"published": true}'
```

Удаление книги:

```
curl -X DELETE http://localhost:3000/api/books/2
```

Ответ: пустое тело, статус 204.

Попытка удалить автора, у которого есть книги (ожидаем 409):

```
curl -X DELETE http://localhost:3000/api/authors/1
```

Ответ: {"error":"Нельзя удалить автора с книгами"} — PostgreSQL защищает целостность данных через внешний ключ.

Проверка дубликата email (ожидаем 409):

```
curl -X POST http://localhost:3000/api/authors \
  -H 'Content-Type: application/json' \
  -d '{"name": "Тест", "email": "tolstoy@example.com"}'
```

Ответ: {"error":"Email уже существует"} — обработка кода Prisma P2002.

404 на несуществующий маршрут:

```
curl http://localhost:3000/nonexistent
```

Ответ: {"error":"Маршрут не найден"}

Если все запросы работают корректно — поздравляем, ваше приложение Express + Prisma готово! Вы можете использовать Prisma Studio для визуального просмотра данных:

```
npx prisma studio
```

7.10 Практика

Все задания ниже расширяют приложение, которое вы создали в разделе 7.9. Выполняйте их последовательно — каждое следующее задание строится на предыдущем.

Задание 7.10.1: Добавление модели Genre (жанр)

Цель: добавить новую модель Genre и связать её с книгами (связь многие-ко-многим).

Что нужно сделать:

1. Добавьте в schema.prisma новую модель Genre:

```
model Genre {
    id    Int    @id @default(autoincrement())
    name  String @unique
    books Book[]
}
```

2. Измените модель Book — добавьте связь с жанрами:

```
    genres Genre[]
```

Prisma автоматически создаст промежуточную таблицу _BookToGenre для связи многие-ко-многим.

3. Создайте миграцию:

```
npx prisma migrate dev --name add_genres
```

4. Создайте файл routes/genres.js с маршрутами:

- POST /api/genres — создание жанра (обработка P2002 для дубликата name).

- GET /api/genres — список всех жанров с количеством книг в каждом.

- DELETE /api/genres/:id — удаление жанра.

5. Подключите маршрут в server.js:

```
import genresRouter from './routes/genres.js';
app.use('/api/genres', genresRouter);
```

6. Измените POST /api/books — при создании книги можно указать жанры:

```
// Тело запроса:
{  "title": "1984", "isbn": "978-5-001",
   "authorId": 1,
   "genreIds": [1, 2]  }
```

Подсказка:

```
const book = await prisma.book.create({
    data: {
        title, isbn, authorId,
        genres: { connect: genreIds.map(id => ({ id })) },
    },
    include: { author: true, genres: true },
});
```

Проверьте:

```
curl -X POST http://localhost:3000/api/genres \
  -H 'Content-Type: application/json' \
  -d '{"name": "Роман"}'

curl http://localhost:3000/api/genres
```

Задание 7.10.2: Вложенное создание автора с книгами

Цель: добавить маршрут, который создаёт автора вместе с его книгами в одном запросе.

Что нужно сделать:

1. Добавьте в routes/authors.js маршрут POST /with-books:

```
// POST /api/authors/with-books
router.post('/with-books', async (req, res, next) => {
    // Ваш код здесь
});
```

2. Маршрут должен принимать такое тело запроса:

```
{
    "name": "Фёдор Достоевский",
    "email": "dostoevsky@example.com",
    "books": [
        { "title": "Преступление и наказание", "isbn": "978-5-001" },
        { "title": "Идиот", "isbn": "978-5-002" }
    ]
}
```

3. Используйте вложенное создание Prisma (create внутри create).

4. Обработайте ошибку P2002 (дубликат email или isbn).

Подсказка:

```
const { name, email, bio, books } = req.body;
const author = await prisma.author.create({
    data: {
        name, email, bio,
        books: { create: books },
    },
    include: { books: true },
});
```

Важно:

Разместите маршрут /with-books ДО маршрута /:id, иначе Express воспримет 'with-books' как параметр id.

Задание 7.10.3: Статистика автора

Цель: добавить маршрут, возвращающий статистику по автору.

Что нужно сделать:

1. Добавьте в routes/authors.js маршрут GET /:id/stats (перед маршрутом /:id).

2. Маршрут должен возвращать:

- totalBooks — общее количество книг автора.

- publishedBooks — количество опубликованных книг.

- avgPages — среднее количество страниц (округлённое).

- Если автор не найден — 404.

Подсказка:

```
const id = Number(req.params.id);
const author = await prisma.author.findUnique({ where: { id } });
if (!author) return res.status(404).json({ error: 'Автор не найден' });

const [totalBooks, publishedBooks, agg] = await Promise.all([
    prisma.book.count({ where: { authorId: id } }),
    prisma.book.count({ where: { authorId: id, published: true } }),
    prisma.book.aggregate({
        where: { authorId: id },
```

```
        _avg: { pages: true },
    }),
]);
```

Ожидаемый ответ:

```
{
    "author": "Лев Толстой",
    "totalBooks": 3,
    "publishedBooks": 2,
    "avgPages": 987
}
```

Проверьте:

```
curl http://localhost:3000/api/authors/1/stats
```

Задание 7.10.4: Сортировка и расширенная фильтрация

Цель: добавить параметры сортировки и расширенной фильтрации в маршруты получения списков.

Что нужно сделать:

1. Измените GET /api/books — добавьте поддержку параметров:

- ?sort=title или ?sort=createdAt — сортировка по полю (по умолчанию createdAt).

- ?order=asc или ?order=desc — порядок сортировки (по умолчанию desc).

- ?minPages=100&maxPages=500 — фильтрация по количеству страниц.

- ?authorId=1 — фильтрация по автору.

2. Измените GET /api/authors — добавьте:

- ?search=толстой — поиск по имени (contains, insensitive).

- ?hasBooks=true — только авторы, у которых есть книги (используйте some).

Подсказка для фильтрации авторов с книгами:

```
const where = {};
if (search) where.name = { contains: search, mode: 'insensitive' };
if (hasBooks === 'true') where.books = { some: {} };
// some: {} означает 'существует хотя бы одна книга'
```

Подсказка для фильтрации книг по страницам:

```
if (minPages || maxPages) {
    where.pages = {};
    if (minPages) where.pages.gte = Number(minPages);
    if (maxPages) where.pages.lte = Number(maxPages);
}
```

Проверьте:

```
curl "http://localhost:3000/api/books?sort=title&order=asc"
curl "http://localhost:3000/api/books?minPages=200&maxPages=1000"
curl "http://localhost:3000/api/books?authorId=1&published=true"
curl "http://localhost:3000/api/authors?search=толстой"
curl "http://localhost:3000/api/authors?hasBooks=true"
```

Задание 7.10.5: Транзакция — передача книг другому автору

Цель: создать маршрут, который атомарно передаёт все книги от одного автора другому и удаляет исходного автора.

Что нужно сделать:

1. Создайте файл routes/transfers.js.

2. Добавьте маршрут POST /api/transfers, который принимает:

```
{
    "fromAuthorId": 1,
    "toAuthorId": 2
}
```

3. Внутри маршрута используйте интерактивную транзакцию:

- Проверьте, что оба автора существуют.

- Обновите authorId всех книг исходного автора на ID целевого автора.

- Удалите исходного автора.

- Если любой шаг не удался — транзакция откатывается.

4. Подключите маршрут в server.js.

Подсказка:

```
const result = await prisma.$transaction(async (tx) => {
```

```
    const from = await tx.author.findUnique({ where: { id: fromAuthorId
} });
    const to = await tx.author.findUnique({ where: { id: toAuthorId }
});
    if (!from || !to) throw new Error('Автор не найден');


    const moved = await tx.book.updateMany({
        where: { authorId: fromAuthorId },
        data: { authorId: toAuthorId },
    });


    await tx.author.delete({ where: { id: fromAuthorId } });
    return { moved: moved.count, deletedAuthor: from.name };
});
```

Проверьте:

Создайте двух авторов с книгами, затем выполните передачу:

```
curl -X POST http://localhost:3000/api/transfers \
  -H 'Content-Type: application/json' \
  -d '{"fromAuthorId": 1, "toAuthorId": 2}'
```

После этого убедитесь, что автор 1 удалён, а его книги теперь принадлежат автору 2:

```
curl http://localhost:3000/api/authors/1   # 404
curl http://localhost:3000/api/authors/2   # книги обоих авторов
```

Задание 7.10.6 (дополнительное): Soft Delete

Цель: вместо физического удаления книг — помечать их как удалённые (soft delete). Это распространённый паттерн в реальных приложениях.

Что нужно сделать:

1. Добавьте в модель Book необязательное поле:

```
    deletedAt DateTime?
```

2. Создайте миграцию:

```
npx prisma migrate dev --name add_soft_delete
```

3. Измените маршрут DELETE /api/books/:id — вместо prisma.book.delete() используйте prisma.book.update() и устанавливайте deletedAt: new Date().

4. Измените все маршруты GET для книг — добавьте условие where: { deletedAt: null }, чтобы удалённые книги не попадали в выборку.

5. Добавьте маршрут PATCH /api/books/:id/restore — для восстановления удалённой книги (установите deletedAt: null).

6. Добавьте маршрут GET /api/books/deleted — для просмотра удалённых книг (where: { deletedAt: { not: null } }).

Проверьте:

```
curl -X DELETE http://localhost:3000/api/books/1        # soft delete
curl http://localhost:3000/api/books                    # книга 1 не
видна
curl http://localhost:3000/api/books/deleted            # книга 1 в
списке
curl -X PATCH http://localhost:3000/api/books/1/restore #
восстановление
```

Заключение

В этом модуле мы изучили:

- Различия между SQL и NoSQL базами данных, когда выбирать PostgreSQL.
- Установку PostgreSQL через Docker и локально (Windows, macOS, Linux).
- Prisma ORM: установку, инициализацию, настройку подключения.
- Схему Prisma: модели Author и Book, типы данных, атрибуты (@id, @unique, @relation, @default, @updatedAt).
- Миграции: создание, применение, просмотр SQL, изменение схемы.
- CRUD-операции с Prisma Client: create, findMany, findUnique, update, delete.
- Фильтрацию: contains, gt, lt, in, AND, OR, NOT, фильтрация по связанным моделям (some, every, none).
- Транзакции: массовые операции ($transaction), интерактивные транзакции, вложенное создание.
- Переменные окружения: .env, --env-file, .env.example, .gitignore.

- Полноценное Express + Prisma приложение: структура проекта, маршруты, обработка ошибок Prisma (P2002, P2003, P2025).

Теперь вы умеете работать и с NoSQL (MongoDB + Mongoose), и с SQL (PostgreSQL + Prisma). В следующих модулях мы добавим аутентификацию (JWT), загрузку файлов и развёртывание приложения.